

MEC:

Ao => What: Minimum angle to be “in zone.”

A1 (variation/drop, degrees) =>

Dynamic role: minimum **rise/fall** around a peak (peak must drop by $\geq A1$ before/after).

Static role: in a static window ($\max - \min \leq A1$).

SO if A1 gets higher : we get dynamic **fewer movements** And **more static time**.

T1 (time window, seconds) =>

Dynamic role: **min spacing** between accepted peaks (close peaks we consider the first only)

Static role: **min duration** where fewer movements (small variations) must last.

SO if T1 gets higher : fewer dynamic movements and **less static time**.

Takt_time : to scale results per hour.

The process :

Inputs:

- `angle_array` , `time_array` , `feedback_frequency` (Hz).
- Thresholds: `Ao` , `A1` , `T1`
- Option: `require_both_drops` for peaks;

Steps:

1. Low-pass filter to remove noise.
2. Find Dynamic peaks :
 - a. only consider regions where angle $\geq Ao$.
 - b. Find local peaks; a peak is **valid** if rise and/or fall around it drops by $\geq A1$ (or both sides if `require_both_drops=True`).
 - c. Enforce **min spacing = T1 seconds** between accepted peaks.
3. Calculate Static time:
 - a. Consider only samples $\geq Ao$
 - b. detect windows where $\max - \min \leq A1$ for $\geq T1$ seconds.
 - c. Sum durations
 - d.
4. Scale to per-hour via takt time

Does it compare consecutive angles?

No.

The peak detector is a tiny state machine:

1. When the signal goes $\geq A_0$, it opens a “segment”.
2. While inside that segment, it keeps the **highest sample so far** as `peak_max`.
3. When the signal **drops below A_0** , it “closes” the segment and evaluates that single peak:
 - `rise = peak_max - min(values before the peak within the segment boundary)`
 - `fall = peak_max - min(values after the peak within the segment boundary)`
 - With `require_both_drops=False`, the peak is accepted if **rise $\geq A_1$ OR fall $\geq A_1$** .
4. Then it applies **spacing** (T1 seconds) across accepted peaks to merge too-close events.